

# Алгоритмы и структуры данных

Конспект лекций

---

ЛЕКТОР

Фамилия И. О.

## Содержание

|   |                                   |   |
|---|-----------------------------------|---|
| 1 | Амортизированный анализ . . . . . | 2 |
|---|-----------------------------------|---|

## 1 Амортизированный анализ

Лекция 1 · 2 сентября 2026

Оценка «худший случай одной операции» бывает слишком грубой: дорогая операция может быть редкой, и платить за неё приходится не на каждом шаге. Амортизированный анализ оценивает не отдельную операцию, а стоимость всей последовательности.

### 1.1 Амортизированная стоимость

#### Определение 1.1 · Амортизированная стоимость

Пусть структура данных обрабатывает последовательность операций  $op_1, \dots, op_m$  с фактическими стоимостями  $c_1, \dots, c_m$ . Число  $\hat{c}$  называется **амортизированной стоимостью** операции, если для любой допустимой последовательности длины  $m$

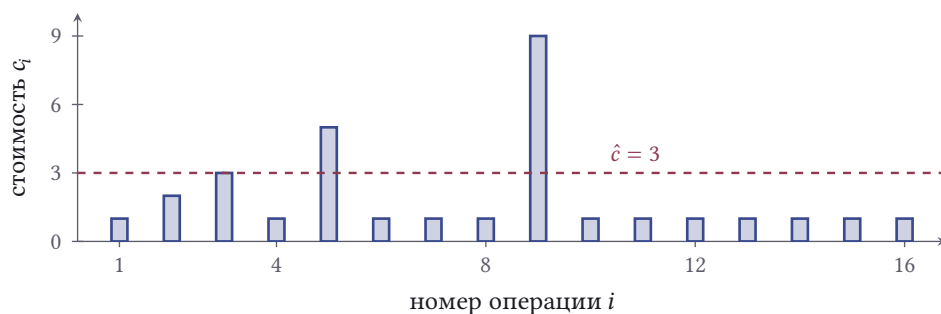
$$\sum_{i=1}^m c_i \leq m \cdot \hat{c}.$$

#### Замечание 1.2

Это не усреднение по вероятности: случайность нигде не используется, оценка обязана выполняться для *худшей* последовательности операций. Отдельная операция при этом может стоить сколь угодно дорого.

### 1.2 Динамический массив

Массив хранит  $n$  элементов в буфере ёмкости  $N$ . Вставка в конец стоит 1, если  $n < N$ ; иначе выделяется буфер вдвое большей ёмкости, старое содержимое копируется, и операция стоит  $n + 1$ .



**Рис. 1.** Фактическая стоимость вставок в динамический массив. Пики приходятся на шаги, где ёмкость удваивается; их вклад покрывается постоянной амортизированной стоимостью.

#### Теорема 1.3 · Динамический массив

Последовательность из  $n$  вставок в конец, начатая с пустого массива ёмкости 1, выполняется суммарно не более чем за  $3n$  элементарных операций. Амортизированная стоимость вставки есть  $\mathcal{O}(1)$ .

**Доказательство.** Разделим стоимость на две части: собственно запись элемента и копирование при расширении.

Записей ровно  $n$ , каждая стоит 1: вклад  $n$ .

Копирование происходит, когда текущий размер равен ёмкости, то есть при размерах  $1, 2, 4, \dots, 2^t$ , где  $2^t \leq n$ . Копируется столько элементов, каков размер, поэтому суммарный вклад копирований равен

$$\sum_{j=0}^t 2^j = 2^{t+1} - 1 < 2 \cdot 2^t \leq 2n.$$

В сумме получаем строго меньше  $3n$ , что и требовалось. Деля на  $n$ , получаем  $\hat{c} < 3$ . ■

⊕  
сумма  
геометрической  
прогрессии

---

**Алгоритм 1:** Вставка в конец динамического массива
 

---

**Input:** массив  $A$  ёмкости  $N$ , размер  $n$ , элемент  $x$

```

1 if  $n = N$  then
2   выделить буфер  $B$  ёмкости  $2N$ ;
3   for  $i \leftarrow 0$  to  $n - 1$  do
4      $B[i] \leftarrow A[i]$ ;
5    $A \leftarrow B$ ;  $N \leftarrow 2N$ ;
6  $A[n] \leftarrow x$ ;  $n \leftarrow n + 1$ ;
```

---

### 1.3 Метод потенциалов

Агрегатный подсчёт из теоремы 1.3 работает, пока последовательность операций однородна. Метод потенциалов снимает это ограничение: он приписывает состоянию структуры число — «запас оплаченной вперёд работы».

#### Определение 1.4 · Потенциал

Пусть  $D_i$  — состояние структуры после  $i$ -й операции. Функция  $\Phi: \{D_i\} \rightarrow \mathbb{R}$  называется **потенциалом**, если  $\Phi(D_0) = 0$  и  $\Phi(D_i) \geq 0$  для всех  $i$ . Амортизированной стоимостью  $i$ -й операции относительно  $\Phi$  называют

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

#### Лемма 1.5

Для любого потенциала  $\Phi$  и любой последовательности из  $m$  операций

$$\sum_{i=1}^m c_i \leq \sum_{i=1}^m \hat{c}_i.$$

**Доказательство.** Сумма телескопируется:

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m c_i + \Phi(D_m) - \Phi(D_0) = \sum_{i=1}^m c_i + \Phi(D_m) \geq \sum_{i=1}^m c_i,$$

поскольку  $\Phi(D_0) = 0$  и  $\Phi(D_m) \geq 0$ . ■

Смысл леммы 1.5 в том, что верхняя оценка на каждое  $\hat{c}_i$  по отдельности автоматически даёт оценку на всю последовательность.

**Пример 1.6 · Двоичный счётчик**

Счётчик хранится как массив битов  $b_0, b_1, \dots$ ; операция INCREMENT переворачивает младшие единицы в нули и следующий ноль в единицу. Стоимость — число перевёрнутых битов — меняется от 1 до  $\Theta(\log v)$ .

| $v$ | $b_3$ | $b_2$ | $b_1$ | $b_0$ | $c_i$ |
|-----|-------|-------|-------|-------|-------|
| 0   | 0     | 0     | 0     | 0     |       |
| 1   | 0     | 0     | 0     | 1     | 1     |
| 2   | 0     | 0     | 1     | 0     | 2     |
| 3   | 0     | 0     | 1     | 1     | 1     |
| 4   | 0     | 1     | 0     | 0     | 3     |
| 5   | 0     | 1     | 0     | 1     | 1     |
| 6   | 0     | 1     | 1     | 0     | 2     |
| 7   | 0     | 1     | 1     | 1     | 1     |
| 8   | 1     | 0     | 0     | 0     | 4     |

Синим отмечены биты, перевёрнутые на текущем шаге. Дорогие шаги ( $v = 4, v = 8$ ) наступают тем реже, чем они дороже.

**Утверждение 1.7**

Амортизированная стоимость INCREMENT равна 2:  $n$  инкрементов переворачивают менее  $2n$  битов.

**Доказательство.** Возьмём в качестве потенциала число единиц в записи счётчика:  $\Phi(D_i) = \#\{b_j = 1\}$ . Условия определения 1.4 выполнены:  $\Phi(D_0) = 0$  и  $\Phi \geq 0$ .

Пусть на  $i$ -м шаге сброшено  $k$  единиц; тогда переворачивается ещё один ноль, так что  $c_i = k + 1$ , а число единиц меняется на  $\Phi(D_i) - \Phi(D_{i-1}) = 1 - k$ . Следовательно,

$$\hat{c}_i = (k + 1) + (1 - k) = 2.$$

По лемме 1.5 суммарное число переворотов не превосходит  $2n$ . ■

**☺ Дополнение: выбор коэффициента роста**

На лекции разбирался только рост в два раза. Тот же подсчёт проходит для любого коэффициента  $\alpha > 1$ : копирования дают  $\sum_j \alpha^j = \mathcal{O}(\alpha^t) = \mathcal{O}(n)$ , поэтому  $\hat{c} = \mathcal{O}\left(\frac{\alpha}{\alpha-1}\right)$  — константа, зависящая только от  $\alpha$ . Практическое значение  $\alpha = 1,5$  (используется, например, в некоторых реализациях `std::vector`) даёт большую константу, но позволяет переиспользовать ранее освобождённую память.

А вот рост на константу  $\Delta$  ломает оценку: расширения происходят на шагах  $\Delta, 2\Delta, \dots$ , и суммарная стоимость копирований равна  $\Theta(n^2/\Delta)$ , то есть  $\hat{c} = \Theta(n)$ .